

第 03 章 | scikit-learn 分類器實務：從直覺觀念到程式應用

學生版公開講義 | 請搭配本課程 Colab，依照段落逐步操作與自我檢查。

來源說明：課程參考《Python Machine Learning, 3rd Edition》；原始範例程式碼採 MIT License。本檔由恩恩統計家教整理為學生版學習摘要，不含原書 PDF 或掃描內容。

上一章從零實作 Perceptron；本章改用 scikit-learn 的一致 API，完成資料前處理、模型訓練、預測與評估。

No Free Lunch Theorem 提醒我們：沒有單一演算法適合所有分類問題。因此，本章比較五種分類器，並依雜訊、線性可分性與特徵維度等資料特性選擇模型。

1. 本章學習路線與核心觀念

本章節在機器學習的地圖中具有「里程碑」式的地位。掌握了 scikit-learn，代表你不再需要重複造輪子，而是能站在巨人的肩膀上，快速調度各種名門演算法來解決真實世界的複雜資料。

核心學習目標

掌握標準工作流：熟悉 scikit-learn 統一的 API 介面，建立「學一通百」的模型訓練 SOP。

深植演算法直覺：理解各類模型（機率型、幾何邊界型、邏輯樹型）的運作邏輯。

診斷並最佳化模型：辨識過擬合（Overfitting）與欠擬合（Underfitting），並運用正規化技術進行調整。

具備實務判斷力：根據問題需求（如可解釋性、運算速度、非線性處理）選擇正確的演算法。

關鍵術語對照表

中文術語	英文術語	說明
訓練集	Training Set	用於讓模型學習規律的資料。
測試集	Test Set	模型未曾見過的資料，用於評估模型的「泛化能力」。
分層抽樣	Stratification	確保訓練集與測試集中的類別比例與原始資料一致。
特徵標準化	Standardization	將特徵縮放至平均值 0，標準差 1，避免數值尺度主宰模型。
準確率	Accuracy Score	預測正確的樣本數佔總樣本數的比例。

確立了實務目標後，第一步我們必須先建立一套萬用的「生產線標準作業程序」。

2. scikit-learn 標準五步工作流 (Standard Workflow)

scikit-learn 將不同演算法包裝成一致的 API。無論使用感知器或隨機森林，都可沿用相同的訓練、預測與評估流程。

核心觀念：機器學習的「工廠生產線」

想像你在經營一家果汁工廠。你不能直接把剛採下來的蘋果（原始資料）丟進罐子裡，你需要經過清洗（前處理）、切片（特徵縮放）、裝瓶（初始化模型）到最後的品檢（評估）。

五步 SOP 導讀

準備與切分資料 (`train_test_split`)：我們必須將資料切分為訓練集與測試集。請記住：拿訓練過的資料來測試，就像拿考過的考古題當期末考，這完全沒有鑑別度。

特徵標準化 (`StandardScaler`)：若一個特徵是年薪、另一個是年齡，兩者數值尺度差距很大；依賴距離或梯度的模型可能因此偏向數值較大的特徵。

初始化模型：宣告模型物件，並設定超參數（如 C 或 γ ）。

模型訓練 (`fit`)：呼叫 `fit(X_train, y_train)` 後，模型會從訓練資料估計參數。

預測與評估 (`predict` / `score`)：讓模型對測試集進行作答，並計算準確率。

為什麼特徵標準化很重要？為避免資料外洩 (Data leakage)，只能用訓練集 `fit` 平均值與標準差，再用同一組參數 `transform` 訓練集與測試集。測試集資訊不能參與前處理參數的估計。

術語對照：常用程式方法

`fit()`：學習規律（如計算平均值、找權重）。

`transform()`：套用學習到的規律進行轉換。

`predict()`：讓模型輸出預測類別。

確立了 SOP 後，接著套用第一個機率式分類模型：邏輯斯迴歸。

3. 邏輯斯迴歸 (Logistic Regression)：機率式分類

邏輯斯迴歸雖以「迴歸」命名，實際用於分類，並可輸出樣本屬於各類別的估計機率。

核心觀念：滑梯 (Sigmoid)

邏輯斯迴歸多了一道「溜滑梯」—— Sigmoid 函數。它會把算出來的原始分數壓縮到 0 到 1 之間。

如果輸出 0.85，代表模型有 85% 的把握說這朵花是鳶尾花。

這在醫學診斷（患病機率）或氣象預報（降雨機率）中非常有價值。

程式碼流程導讀

```
from sklearn.linear_model import LogisticRegression

# 1. 初始化模型
# 注意：solver 在新版 scikit-learn 預設為 'lbfgs'，它比舊版的 'liblinear' 更適合處理多元分類
lr = LogisticRegression(C=100.0, solver='lbfgs', multi_class='ovr')

# 2. 訓練模型
lr.fit(X_train_std, y_train)

# 3. 預測類別與機率
y_pred = lr.predict(X_test_std) # 給出類別答案 (如 0, 1, 2)
y_prob = lr.predict_proba(X_test_std) # 給出機率陣列，每橫列加總必為 1.0
```

差別在哪？`predict()` 會根據機率最大值直接給你標籤；而 `predict_proba()` 則讓你看到模型內心的機率分布（Conditional Probability）。

術語對照

Sigmoid Function (S 型曲線)：將輸入映射到 (0, 1) 區間的激活函數。

Conditional Probability (條件機率)：在給定特徵的情況下，樣本屬於某一類別的機率。

模型訓練完後，我們必須診斷它是否「學得太死」或「學得太少」。

4. 診斷模型：過擬合、欠擬合與正規化 (Regularization)

訓練模型就像準備考試，我們必須在「偏差 (Bias)」與「變異 (Variance)」之間取得平衡。

核心觀念：死背考古題 vs. 完全不讀書

欠擬合 (Underfitting / High Bias)：就像完全沒唸書，模型太簡單，連考古題都考不及格。

過擬合 (Overfitting / High Variance)：就像走火入魔，把考古題連同錯字全部死背。平時考 100 分，但大考題目稍微變一下就崩潰。

實際意義：C 如何控制過擬合

在 scikit-learn 中，我們透過參數 C 來控制正規化強度。這是一個歷史慣例，繼承自 SVM。

C 是正規化強度的「倒數」($C=1/\lambda$)。

當 C 變小：對權重的「懲罰」增加，模型會被迫變得簡單平滑，藉此防止過擬合。

當 C 變大：模型會放任權重增長，極力追求減少訓練錯誤，容易導致過擬合。

在 SVM 的脈絡下，C 也代表「軟邊界 (Soft-margin)」的錯誤容忍度。

掌握了預防過擬合的工具後，我們來看另一種追求「最大化馬路寬度」的強悍分類器。

5. 支持向量機 (SVM) 與核技巧 (Kernel Trick)

SVM 透過最大化類別間隔 (Maximum Margin) 尋找決策邊界；間隔越大，通常越能容忍新資料的小幅變動。

核心觀念：最大間隔與核技巧

SVM 基礎：想像你要在紅、藍兩群資料中開一條路。SVM 要求這條「馬路」越寬越好。決定這條路寬度的關鍵點，就是卡在路邊的點，我們稱之為「支持向量 (Support Vectors)」。

核技巧 (Kernel Trick)：同心圓資料無法在原始 2D 空間以直線分開。核函數可隱式計算較高維空間中的相似度，使 SVM 建立非線性決策邊界。

重要參數解析： γ (Gamma)

γ 決定了單一訓練樣本的「勢力範圍」。

γ 越大：每個點的勢力範圍越小，決策邊界會緊緊包繞每個點，邊界變得扭曲且支離破碎（極易過擬合）。

γ 越小：勢力範圍大，決策邊界會變得比較平滑、穩定。

術語對照

Support Vectors (支持向量)：最接近決策邊界的關鍵訓練樣本。

RBF Kernel (徑向基底函數核)：最常用的非線性核函數，能處理複雜的邊界。

若需要能直接說明判斷規則的模型，可以考慮決策樹。

6. 基於樹的演算法：決策樹 (Decision Trees) 與隨機森林 (Random Forest)

在業界，模型的可解釋性 (Interpretability) 有時比準確率更重要。

核心觀念：猜猜我是誰與集體決策

決策樹：就像「猜猜我是誰」遊戲。模型會自動找問題（如：花瓣寬度 > 0.75 嗎？），不斷切分直到得出答案。

隨機森林：這是「集成學習 (Ensemble Learning)」的代表。既然一棵樹容易出錯，我們就種 100 棵樹，讓它們「投票」決定答案。這大幅提升了模型的穩健性。

為什麼決策樹不需要特徵縮放？

判讀重點：決策樹關心的是「在哪個數值切開最好」，它看的是資料的數值順序與等級，而不是數值的比例。因此，它是少數完全不需要做 `StandardScaler` 的模型。

程式碼流程導讀

```
from sklearn.tree import DecisionTreeClassifier

# max_depth 控制樹的生長。樹太深會導致過擬合（死背細節）
tree = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=1)
tree.fit(X_train, y_train) # 直接輸入原始特徵即可
```

術語對照

Information Gain (資訊增益)：切分後資料變得多純粹的指標。

Gini Impurity (吉尼不純度)：衡量節點的混亂程度，數字越小越好。

最後我們看一個「最懶惰」卻最直覺的模型。

7. K-近鄰演算法 (KNN)：物以類聚的懶惰學習

KNN 堪稱演算法界的「邊緣人」，它在訓練階段什麼都不學，只是把資料背下來。

核心觀念：看鄰居寫答案

KNN 的核心就是「物以類聚」。當新樣本進來時，它就在空間裡找離它最近的 K 個鄰居。

scikit-learn 的平手邏輯：如果鄰居投票平手，它通常會選擇距離較近的那一方，或依照原始資料的排序。

關鍵警示：維度詛咒 (Curse of Dimensionality)

因為 KNN 完全依賴「距離」計算，如果特徵維度太高，空間會變得無窮大。原本的「鄰居」其實也離你幾十萬光年遠。這時 KNN 會完全失效。此外，特徵標準化對 KNN 是絕對前提，否則尺標大的特徵會支配整個距離計算。

術語對照

Lazy Learner (懶惰學習)：不在訓練期學習參數，而是把計算推遲到預測期。

Minkowski Metric (閔考斯基距離)：歐幾里得距離與曼哈頓距離的統稱。

8. Google Colab 實戰指南與常見錯誤排除

Colab 操作步驟

由於畫圖套件依賴系統層級軟體，請先執行指令：

```
# 安裝系統層級的圖表繪製引擎與套件
!apt-get install graphviz
!pip install pydotplus
```

讀取 Iris 資料集：

```
from sklearn.datasets import load_iris
iris = load_iris()
X, y = iris.data[:, [2, 3]], iris.target
```

常見錯誤與排除清單

錯誤訊息 / 警告	可能原因	修復方案
ConvergenceWarning	LogisticRegression 迭代次數不足	手動指定 solver='lbfgs' 並增加 max_iter。
FileNotFoundError	系統找不到 Graphviz	請確認在 Colab 執行了 !apt-get install graphviz。
準確率意外極低 (SVM/KNN)	忘了做特徵標準化	請檢查 SOP 的第二步是否確實執行 StandardScaler。

動手練習：模型選擇決策邏輯樹

你可以根據以下條件選擇模型：

是否需要向非技術利害關係人說明判斷規則？

是 -> 決策樹

否 -> 進入下一步

資料是否非常混亂、非線性？

是 -> SVM (RBF Kernel) 或 隨機森林

否 -> 進入下一步

是否需要預測精確的機率（如 85% 降雨）？

是 -> 邏輯斯迴歸

否 -> 進入下一步

資料量極小且特徵單純？

是 -> KNN (記得一定要做標準化！)

自我檢查表：

() 在 **LogisticRegression** 中，為了減輕過擬合，我應該調小 C。

() SVM 使用 RBF 核時， γ 設太大會導致模型只看局部，容易過擬合。

() 隨機森林是透過多棵決策樹的「多數決」來降低單一樹的變異。

完成本章後，請開啟 Colab，依序執行分類器範例並比較決策邊界。